ERA · AI CLINIC ASSISTANT

Era — Features & Working Flows

AI Face & Voice Clinic Assistant · Version 4.3

Compiled from the v4.3 codebase (server.js, lib/*.js, app.js, index.html) · October 5, 2026

Contents

1. **Era — Features & Working Flows**
    - System at a glance
    - Part 1 — Features list with implementation
    - Part 2 — Full working flows
    - Appendix — API endpoint map

Era — Features & Working Flows

Era is an AI face & voice clinic assistant: a single-page vanilla-JS browser app plus a **zero-npm-dependency Node ≥ 22.5 server** with SQLite (`node:sqlite`), which speaks every reply through **NVIDIA NVCF** in one voice — **Sofia** (English / हिन्दी / Hinglish, picked automatically). This document lists every feature with its implementation, then walks through the complete working flows of the system. *(v4.3 · compiled from the codebase · October 2026)*

System at a glance

Piece	Technology	Where
Client	Single-page vanilla JS, Manrope font, photo character, voice-first UI	`index.html`, `app.js`, `style.css`, `assets/era3d.png`
Server	Node built-ins only (`node:http`, `node:sqlite`, `node:fs`, `node:crypto`)	`server.js` + `lib/*.js`
Database	SQLite via `node:sqlite` `DatabaseSync`, FK enforcement, 10 tables	`lib/database.js` → `db/era.db`
Knowledge base	15-row AI-doctor Q&A + word-boundary keyword scoring (dual browser/Node module)	`db/db.js`
Voice	NVIDIA NVCF Magpie-Multilingual — Sofia EN-US + HI-IN only	`lib/tts.js`, `tts-grpc.py` (gRPC bridge)
Auth	scrypt password hashing + HS256 JWT (8 h), Bearer or HttpOnly cookie	`lib/auth.js`
Config	gitignored `.env` loader (file wins over shell env), `nvcf.key`, optional certs	`lib/env.js`
Tests	18-check security suite on an isolated scratch DB	`test/security.test.js`

Module map (v4.2 split): `env → auth → database → tools → brain → tts → router`; `server.js` is a thin entry point that owns listeners and the `__test` export surface only.

Part 1 — Features list with implementation

1. Identity & access control

Feature	What it does	Implementation
One-click demo login	Signs in as Dr. Amit / Dr. Rahul / Dr. Priya / Admin / Front Desk without typing	`POST /api/assistant/demo-login` (lib/router.js): guarded by the `DEMO_LOGIN` switch (lib/env.js, default ON; `DEMO_LOGIN=0` → endpoint 404s). Maps `who` → seeded user name, mints a JWT, audited as `demo_login`
Email sign-in **and** auto sign-up	One form logs in existing users and registers new ones	`POST /api/auth/login` (lib/router.js): valid email + password ≥ 4 chars; known email → scrypt verify → 200; unknown email → creates `users` (role `doctor`) + `doctors` row → 201 `created:true`; wrong password → 401 with a helpful hint

Feature	What it does	Implementation
Password hashing	Passwords never stored plain	`crypto.scryptSync(pw,'era-salt-v1',32)` hex + `timingSafeEqual` compare (lib/auth.js) — demo-grade salt, documented swap path to bcrypt/argon2
JWT sessions	Stateless 8-hour tokens	`mintToken` / `verifyJWT` (lib/auth.js): HS256 HMAC, base64url payload `{sub, role, doctorId, name, iat, exp}`; accepted from `Authorization: Bearer` **or** the HttpOnly `era_token` cookie; signature + expiry checked on every request
Role model	`doctor` / `admin` / `reception`	`requireAuth`, `requireRole(roles)`, `requireDoctor` middleware (lib/auth.js); friendly denial texts, never raw internals
Doctor scoping	Every query pinned to the authenticated doctor	`scopeDoctor` (lib/tools.js): doctors are pinned to their own `doctorId`; **only admin** may pass an explicit `doctor_id` (validated integer); AI/frontend-supplied ids never widen scope
Session restore	Reload keeps you signed in	Client `loadMe()` → `GET /api/assistant/me` → user + doctor profile → straight into the workspace
Logout	Ends the session cleanly	`POST /api/auth/logout` clears the cookie (audited); client clears the greeting flag, aborts audio + mic
Test-token CLI	Mint JWTs for API testing	`token.js`, driven through the preserved `__test` export surface of server.js

2. AI assistant chat (the brain)

Feature	What it does	Implementation
Chat endpoint	The single answering path	`POST /api/assistant/chat` (lib/router.js `chat()`): requireAuth → direct tool call *or* `routeIntent` → friendly denial path for non-doctors → `runTool` → `sanitize` → `toolReply` wording → audit
Intent routing	Maps natural language to tools	`routeIntent(msg, user)` (lib/brain.js): next appointment · cancel `#id` · book/create · meetings · appointments (today/tomorrow) · working hours / available slots · schedule / agenda / my day · what's next · patients · prescriptions · profile · health KB · greeting · fallback
Shared-schedule answers	Admin/reception (no doctorId) still get day answers	brain.js answers meeting/schedule/next questions from the shared `schedule` table (`clinicDay` / `nextClinic`) — clinic-wide data, safe for every role
Health knowledge base	15 curated first-aid/lifestyle rows	`ai_doctor` table; `EraDB.scoreRows` (db/db.js): phrase substring +2, single-word exact/prefix ≥ 4 chars +1; highest score wins, 0 → miss; keywords stored as JSON string in SQL, array in seed
Human-sounding replies	Tool rows become spoken sentences	`toolReply` (lib/router.js): friendly dates (`today` / `tomorrow` / weekday), 12-hour TTS-friendly times, bulleted lists; write confirmations ("Booked your appointment (#12).")
Data minimization	Internal keys never leave the server	`sanitize()` strips `doctor_id` / `patient_id` from every `tool_result` before it reaches client or AI
Audit trail	Every chat/tool/login/denied/error is recorded	`assistant_audit(user_id, doctor_id, action, tool_name, request_id, success, detail, resource, resource_id)`; no message bodies, no secrets

Feature	What it does	Implementation
Safe errors	No SQL, stack traces, or enumeration	Uniform 401 "Please sign in…", 403/404 "I can only access information available to your RMS account."; ownership failures are 404 (indistinguishable from *not exists*); every response carries a `request_id`

3. RMS data layer

Feature	What it does	Implementation
SQLite engine	Real SQL file database, zero driver install	`node:sqlite` `DatabaseSync` ; `PRAGMA foreign_keys = ON` per connection; one `ExperimentalWarning` expected
RMS schema	Multi-user clinic model	`users, doctors, patients, doctor_patients, doctor_schedules, appointments, prescriptions, assistant_audit` + `ai_doctor` + `schedule` ; FKs, CHECKs, timestamps, 7 indexes on doctor-scoped fields
Ownership pattern	Doctor-level isolation in SQL	Every read/write includes `AND doctor_id=?` in the WHERE clause — never `WHERE id` alone
REST resources	Dashboard-ready API	`GET/POST /api/appointments` (filters `date/from/to/status/upcoming` , LIMIT 100) · `GET/PATCH/DELETE /api/appointments/:id` · `GET /api/schedule` · `GET /api/patients` (LIMIT ≤ 50 + OFFSET) · `GET /api/patients/:id` · `GET/POST/PATCH/DELETE /api/prescriptions[/:id]`
AI tools	Read + write through one permission path	lib/tools.js: `getMyAppointments, getMySchedule, getMyPatients, getAppointmentDetails, getAvailableSlots, getMyProfile, getMyPrescriptions` + writes `createAppointment, rescheduleAppointment, cancelAppointment` — each re-validates ownership; slot clash → 409; foreign patient → 403
Smart slot search	Free 30-minute slots per working day	`getAvailableSlots` : walks `doctor_schedules` windows for the weekday, subtracts taken times, defaults 30-min length (`addMin`)
Idempotent seeding	Self-healing demo data	`ai_doctor` upserted from `db/db.js` on **every** start (source of truth); `schedule` seeded only when empty; on empty `users` : 5 users (3 doctors with specialties, Admin, Front Desk — `password123` hashed), 5 patients + linkages, weekly schedules, 4 demo appointments, 2 prescriptions
Demo freshness	"Tomorrow" never answers empty	`APPOINTMENT …` demo rows are re-dated — first to **today**, rest to **tomorrow** — at every server start
Additive migrations	Old DBs upgrade in place	`ALTER TABLE` for `users.password_hash` , `appointments.patient_name` , audit `resource` / `resource_id` ; tolerant of "duplicate column" errors, never destructive
Cloud fallback chain	Boots anywhere (Render free tier)	lib/database.js tries `ERA_DB` path → repo-local `db/era.db` → `os.tmpdir()/era.db` , `mkdirSync(recursive)` each, logs `[era] SQLite: <path>`

4. Voice pipeline (NVIDIA NVCF)

Feature	What it does	Implementation
TTS proxy	The API key never reaches the browser	`POST /api/tts` (JWT required, lib/router.js) → `synthesize` (lib/tts.js) → NVCF; WAV streamed back as `audio/wav`

Feature	What it does	Implementation
One voice: Sofia	Single female voice, two languages	`SOFIA_VOICES = ['Magpie-Multilingual.EN-US.Sofia','Magpie-Multilingual.HI-IN.Sofia']` — **single owner** in lib/env.js; the voices list filters to it and `POST /api/tts` rejects anything else with `400 BAD_VOICE`
Auto-language	Hindi/Hinglish replies speak in हिन्दी	Client `pickVoice(text)` (app.js): Devanagari regex or romanised-Hinglish word-list → `HI-IN.Sofia`; otherwise the selected/EN Sofia
Proper-case language tags	Avoids Triton "Mapping failed"	`language` derived from the voice name then re-cased `en-US` / `hi-IN` (uppercase `EN-US` breaks synthesis — root cause of past live failures)
Dual-transport retry	Survives NVCF flakiness	3× HTTP multipart (`text/language/voice/sample_rate_hz=22050/LINEAR_PCM`, 60 s timeout, 0.8/2 s backoff) → 2× gRPC via `python tts-grpc.py` (90 s timeout, raw PCM wrapped in a 44-byte WAV header); every payload validated as real RIFF before serving/caching; every attempt logged into `errs[]`
WAV cache	Repeats instant, upstream down-tolerant	LRU Map keyed `voice+\ntext`, cap 20, refreshed on hit
Voice list	Settings dropdown, filtered	`GET /api/tts/voices` (JWT): fetches NVCF list, filters to Sofia EN+HI, marks default; 503 when no key, 502 when upstream down; client falls back to a built-in two-option list
Spoken replies	Voice-only interface	Client `speak()`: `POST /api/tts` with `pickVoice`, 65 s timeout → blob → `<audio>` object URL; failure → status "Voice service unavailable" — **no** browser-voice fallback by design (v3.9 removal)
Playback guard	Stop/new-reply/logout kill audio instantly	`speakSeq` generation token + `stopSpeak()` (pauses, revokes object URL, resets mouth); mic press stops current audio first
Mic input	Hands-free questions	Web Speech `SpeechRecognition` (`en-IN`), feature-detected; unsupported → spoken hint + slim typed input

5. Client & UI

Feature	What it does	Implementation	
Two-view flow	Auth card ↔ workspace	`show('auth'	'app')` `toggles` .hidden `; startup` loadMe()` decides (flash-of-content guard)
Pop-up greeting	Personalized, once per login session	`sessionStorage.era_greeted` (cleared on logout): time-of-day + name → "Good morning, Dr. Amit! I'm Era…" after 400 ms; refresh = quiet restore	
Photo character	3D-look clinician is the interface	`assets/era3d.png` in `#photoWrap`, CSS `float` / `breathe` idle + `talk` pulse on `[data-state=speaking]`; framing knobs `--eraPhotoW/H/X/Y`; status pill `#status` (Ready / Listening… / Speaking… / Thinking…), pulsing rings, wave bars	

Feature	What it does	Implementation
Voice-first UX	Talk button, spoken replies, no chat panel	Mic → `ask(transcript)`; one slim text input as fallback (Enter/Send); replies update `#live` (`.sr-only`, `aria-live=polite`) so they stay accessible when TTS is off
Settings dialog	Name, speak/greet toggles, NVIDIA voice	`<dialog id=set>`: native modal; voice select filled from `/api/tts/voices` (Sofia EN/हिन्दी); saved to `localStorage.era_settings` via try/catch-wrapped helpers
Accessibility	Keyboard + screen-reader friendly	`:focus-visible` outlines, `title` tooltips, `prefers-reduced-motion` kills animations, hidden live region
Fresh assets	Stale-JS protection	Static files served with `Cache-Control: no-cache`; scripts cache-busted `?v=4.0.0`
Embedding CORS	Ready for a dashboard origin	CORS headers locked to the dashboard origin (`http://localhost:5500`) with credentials; OPTIONS → 204; request body capped at 100 KB

6. Ops, configuration & verification

Feature	What it does	Implementation
`.env` configuration	One source of truth per process	lib/env.js parses gitignored `.env`, **file wins over shell env** (defeats box-wide junk like `PORT=0`); keys: `PORT` (8000), `HTTPS_PORT` (8443), `ERA_DB`, `JWT_SECRET`, `DEMO_LOGIN`, `NVIDIA_TTS_URL`, `NVIDIA_TTS_VOICE`, `NVIDIA_API_KEY`
Secret handling	Keys never committed/logged	`NVIDIA_API_KEY` env or gitignored `nvcf.key` read server-side only (lib/tts.js), never logged, never sent to the client
HTTPS mirror	Local TLS for mic-friendly testing	When gitignored `certs/key.pem` + `cert.pem` exist → second listener on 8443 sharing the same `handler`
Watchdog	Server always comes back	`start-era.cmd` (CRLF, absolute path) restarts `node server.js` every 3 s; optional logon Scheduled Task
Thin entry + test seam	Clean imports for tests	`server.js` listens only when `require.main === module`; exports `__test` (auth, tools, runTool, scopeDoctor, routeIntent, db, EraDB, audit, sanitize, toolReply)
Security suite	18 checks green	`test/security.test.js` on a scratch DB (`ERA_DB` env, set before require): auth, roles, scoping, ownership, audit, sanitization
Render deploy	Live at `https://era-assistant.onrender.com`	build `npm test`, start `npm start`; `ERA_DB=/var/data/era.db`, `DEMO_LOGIN=1`, `JWT_SECRET` + `NVIDIA_API_KEY` via dashboard; free-tier disk is ephemeral → DB re-seeds itself after redeploys (fallback chain prevents crash loops)

Part 2 — Full working flows

Flow 1 — Startup & seeding

1. `node server.js` requires **lib/env.js first** → gitignored `.env` parsed, file values overwrite shell env.

2. **lib/database.js** opens SQLite through the fallback chain (`ERA_DB` → `db/era.db` → temp dir), runs `PRAGMA foreign_keys=ON`, logs the chosen path.
3. Schema is created `IF NOT EXISTS` (10 tables + 7 indexes); additive migrations upgrade pre-v3.1 databases.
4. Seeding: `ai_doctor` upserted from `db/db.js`; `schedule` and `users` seeded only when empty (demo clinic: doctors, patients, linkages, schedules, prescriptions).
5. Demo `APPOINTMENT …` rows re-dated to today/tomorrow so voice queries always have data.
6. **server.js** starts HTTP on `PORT`; optional HTTPS mirror on 8443; `__test` exports attached; the app serves at `http://localhost:8000` (static map: index.html, app.js, style.css, db/db.js, era.png, era3d.png).

Flow 2 — Sign-in (three paths)

- **Demo:** persona select → `POST /api/assistant/demo-login {who}` → JWT minted + `Set-Cookie: era_token (HttpOnly, SameSite=Lax)` → workspace.
- **Email:** form submit → `POST /api/auth/login` → 200 "Welcome back" / 201 "Account created" / 400-401 with message → `ME` stored → workspace.
- **Session restore:** page load → `GET /api/assistant/me` → 200 → straight to workspace; otherwise the auth view shows.

Flow 3 — Pop-up greeting (once per login session)

`showApp()` → checks `sessionStorage.era_greeted` → if unset and "Greet me" is on: flag set, greeting composed from the clock + authenticated name, spoken after 400 ms. Refreshes restore quietly; logout clears the flag.

Flow 4 — The core voice loop (ask → answer → speak)



Step details:

1. **Auth first:** identity comes only from the JWT; a 401 mid-session makes the client say "Your session expired" and return to the auth view.
2. **Intent ladder** (first match wins): "next appointment" → newest scheduled row beyond now; "cancel 12" → write tool with id; "book/add … appointment" → create flow; "meetings" → shared `schedule`; "appointments tomorrow" → dated tool call; "available slots/free" → slot generator; "patients/prescriptions/my profile" → their tools; **health phrasing falls through to the knowledge base**; greeting/fallback texts keep the assistant helpful.
3. **Tool execution:** `scopeDoctor` pins the doctor (admin may pass an explicit `doctor_id`); all SQL parameterized; 404 for foreign rows, 403 for foreign patients, 409 for clashes.
4. **Reply shaping:** `toolReply` converts rows to spoken prose; `sanitize` strips internal ids; the audit row records user, doctor, action, tool, request id.
5. **Speech:** the client picks Sofia EN or HI automatically, the server runs the cache → 3×HTTP → 2×gRPC ladder, validates RIFF, caches, and streams WAV; the `speakSeq` token makes Stop/new-reply/logout silence the speaker instantly.

Flow 5 — Health question (knowledge base)

"i have a fever" matches no tool intent → `EraDB.scoreRows` scores all 15 `ai_doctor` rows (substring +2, word-boundary/prefix +1) → best match returns generic guidance (urgent topics always include a see-a-doctor pointer) → spoken in Sofia. "what is the weather" scores 0 → honest fallback message, never a false health answer.

Flow 6 — Write actions (book / reschedule / cancel)

1. Voice: "add a meeting called Review at 3 pm" or "cancel appointment 12" → write intent → tool runs with the authenticated doctor's id.
2. `createAppointment`: requires patient (linked id **or** name), date, time → patient-linkage check (403) → slot-clash check (409) → INSERT with 30-minute default → "Booked your appointment (#13)."
3. `rescheduleAppointment`: ownership check → clash check excluding itself → UPDATE.
4. `cancelAppointment`: soft-cancel (`status='cancelled'`) with ownership in the WHERE; nothing matched → 404.
5. Every write is audited with `resource`/`resource_id`; confirmation chips before firing writes are on the backlog (R4).

Flow 7 — Admin & Front Desk experience

Users without a `doctorId` get **shared-schedule answers** ("3 meetings and 2 appointments today…") from the clinic `schedule` table. Asking for doctor-owned data returns a friendly `200 {denied:true}` with guidance — the tool layer still enforces security underneath; no raw 403 console noise. Admin can still call tools directly with an explicit `doctor_id` (validated by `scopeDoctor`).

Flow 8 — Embedding into a real RMS dashboard

1. Dashboard authenticates (its own login or Era's), holds the JWT.
2. REST: `GET /api/appointments?upcoming=1`, patients with pagination, prescriptions CRUD — all doctor-scoped server-side; CORS is pre-locked to the dashboard origin with credentials.
3. Or drive the assistant directly: `POST /api/assistant/chat {tool, args}` (dashboard buttons) — same authorization as voice.
4. Swap-in for production: replace `verifyJWT` with the real RMS auth, drop the demo seed, point CORS at the dashboard origin, set `DEMO_LOGIN=0`.

Flow 9 — Degradation & recovery matrix

Situation	Behavior
NVCF upstream flaky (400 "Mapping failed", timeouts)	3× HTTP → 2× gRPC → LRU cache; if all fail → 502 → Era shows "Voice service unavailable", stays silent until recovery (no browser-voice fallback by design)
No NVIDIA key configured	503 `TTS_NOT_CONFIGURED`; voices list degrades to the built-in Sofia-only dropdown
Wrong voice requested	400 `BAD_VOICE` — Sofia-only enforced at the endpoint, not just the dropdown
Language tag casing	Always re-cased to `en-US` / `hi-IN` (uppercase voice-name casing breaks Triton)
`ERA_DB` directory missing / read-only (Render)	Fallback chain: repo `db/` → OS temp dir; app still boots and re-seeds
Chat API unreachable	Client: "I cannot reach the clinic server right now…" (chat answers are server-only by design; the raw `/api/qa` endpoint keeps a bundled offline fallback via `db/db.js`)
Session expired (401)	"Your session expired — please sign in again." + automatic return to auth view
Mic unsupported (Firefox/Safari)	Spoken hint + the slim typed input
Slot already taken	409 → "That time slot is already booked." (spoken)
Stale client code	Static served `no-cache` + `?v=4.0.0` cache-bust

Flow 10 — Verification & deployment

- `npm test` → **18/18** on an isolated scratch DB (`ERA_DB` set before requiring the server).
- `node --check` per file; `GET /api/health` shows row counts; `GET /api/qa?q=fever` hits, `q=weather` misses (boundary check).
- Local run: `node server.js` (or `start-era.cmd` watchdog) → http://localhost:8000 + optional https://localhost:8443.
- Render: build `npm test`, start `npm start`; `ERA_DB=/var/data/era.db`, `DEMO_LOGIN=1`, `JWT_SECRET` and `NVIDIA_API_KEY` set in the dashboard; free tier sleeps after ~15 min (wake ~30-50 s) and the ephemeral disk means the DB re-seeds after each redeploy.

Appendix — API endpoint map

Method & path	Auth	Purpose
`GET /api/health`	—	Liveness + row counts + version
`GET /api/qa?q=…`	—	Raw knowledge-base lookup (scored)
`POST /api/auth/login`	—	Sign-in **or** auto sign-up → JWT + cookie
`POST /api/auth/logout`	optional	Clears the session cookie (audited)
`POST /api/assistant/demo-login`	`DEMO_LOGIN=1`	One-click demo session (Amit/Rahul/Priya/Admin/Front Desk)
`GET /api/assistant/me`	JWT	Current user + doctor profile
`POST /api/assistant/chat`	JWT	Message **or** direct tool call → reply + tool_result
`GET /api/tts/voices`	JWT	Sofia-filtered NVCF voice list

Method & path	Auth	Purpose
`POST /api/tts`	JWT	Text → WAV (cache + dual-transport retry)
`GET /api/appointments`	doctor	List with `date/from/to/status/upcoming` filters
`POST /api/appointments`	doctor	Create (patient link check, clash → 409)
`GET/PATCH/DELETE /api/appointments/:id`	doctor	Read/update/cancel, ownership in WHERE
`GET /api/schedule`	doctor	Weekly working hours
`GET /api/patients` · `GET /api/patients/:id`	doctor	Paginated patient list / detail (cross-doctor → 404)
`GET/POST/PATCH/DELETE /api/prescriptions[/:id]`	doctor	Prescriptions CRUD, doctor-scoped